Antes de acelerar, ¿qué debemos calcular?

Fundamentos, referencia secuencial y primeras mediciones

Proyecto universitario de computación de alto rendimiento · 21 de septiembre de 2026

El objetivo del proyecto es construir y evaluar una factorización de Cholesky paralela. Comenzamos por una versión secuencial cuyo resultado podamos verificar. Cada avance seguirá una pregunta, una hipótesis, una implementación y un experimento; los resultados decidirán la siguiente mejora.

1. La pregunta inicial

Dada una matriz real A, simétrica y definida positiva (SPD), buscamos una matriz triangular inferior L con diagonal positiva que cumpla:

$$A = LL^T$$

Simétrica significa que A coincide con su transpuesta. Definida positiva significa que su forma cuadrática es positiva para cualquier vector real no nulo:

$$A = A^T, \qquad x^T Ax > 0 \quad \text{para } x \neq 0$$

La factorización sirve, por ejemplo, para resolver un sistema Ax = b: primero se resuelve Ly = b y después L^Tx = y. En esta etapa implementamos solamente la factorización.

2. Qué esperamos observar

Hipótesis: si la matriz es SPD y respetamos las dependencias del algoritmo, el producto de L por su transpuesta reconstruirá A, salvo los errores de redondeo de la aritmética de punto flotante.

En aritmética exacta, las condiciones SPD garantizan los pivotes positivos y la existencia de esta factorización. En una computadora verificaremos los pivotes y los resultados no finitos: una entrada mal condicionada puede presentar dificultades numéricas.

3. Alcance de esta primera etapa

Usamos Python como referencia didáctica porque permite relacionar directamente las ecuaciones con el código. No representa una implementación HPC optimizada. La referencia de rendimiento para MPI se construirá en C; no calcularemos speedup comparando Python con C.

De las ecuaciones a una columna de L

Usamos índices desde 0, como en el programa. Al expandir el producto matricial, la triangularidad limita la suma:

$$a_{ij} = \sum_{k=0}^{\min(i,j)} l_{ik} l_{jk}$$

4. Primero la diagonal; después, debajo de ella

Para la columna j, los términos de las columnas anteriores ya son conocidos. Despejamos primero el elemento diagonal y luego cada elemento inferior:

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=0}^{j-1} l_{jk}^2}$$

$$l_{ij} = \frac{a_{ij} - \sum_{k=0}^{j-1} l_{ik} l_{jk}}{l_{jj}}, \qquad i > j$$

Cuando j = 0, la suma vacía vale cero. Los elementos bajo la diagonal de una misma columna pueden calcularse de forma independiente una vez disponibles el pivote y las columnas anteriores. Esta observación será útil al estudiar paralelismo, aunque no determina por sí sola una estrategia eficiente.

5. Un ejemplo que podemos resolver a mano

$$A = \begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix} \qquad L = \begin{bmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{bmatrix}$$

$$l_{00} = \sqrt{4} = 2, \qquad l_{10} = 12/2 = 6, \qquad l_{20} = -16/2 = -8$$

$$l_{11} = \sqrt{37 - 6^2} = 1, \qquad l_{21} = \frac{-43 - (-8)\cdot 6}{1} = 5$$

$$l_{22} = \sqrt{98 - (-8)^2 - 5^2} = 3$$

El producto LL^T recupera la matriz A del ejemplo. El trabajo aritmético dominante de Cholesky denso es aproximadamente $n^3/3$ operaciones de punto flotante; el almacenamiento denso crece como n^2. En Python también intervienen el intérprete y la representación de los objetos.

Cómo comprobamos que el resultado es correcto

6. Decisiones de implementación

La función **cholesky()** conserva A y construye una L nueva, con ceros por encima de la diagonal. Esto facilita la comparación y evita factorizar una entrada modificada en la siguiente repetición. La referencia exige una matriz cuadrada, no vacía, finita y exactamente simétrica.

En cada paso comprobamos que el pivote sea positivo y finito. También rechazamos resultados no finitos. Una ejecución que no supera estas comprobaciones no se acepta como medición válida.

7. Generación reproducible de matrices SPD

Generamos valores uniformes entre −1 y 1 fuera de la diagonal y los reflejamos para asegurar simetría. Luego fijamos:

$$a_{ii} = 1 + \sum_{j \neq i} |a_{ij}|$$

La diagonal es positiva y estrictamente dominante. Por simetría, los autovalores son reales; los intervalos de Gershgorin quedan en el semieje positivo, por lo que la matriz es SPD. La semilla permite reproducir la entrada. Esta familia inicial no cubre todos los problemas ni matrices arbitrariamente mal condicionadas.

8. Medida de reconstrucción

$$r = \frac{\|A - LL^T\|_F}{\|A\|_F}, \qquad \|M\|_F = \sqrt{\sum_i \sum_j m_{ij}^2}$$

Usamos el residuo relativo para comparar el error con la magnitud de A. Para estas matrices pequeñas adoptamos $r < 10^{-12}$ como criterio práctico. No es un umbral universal: un residuo pequeño tampoco asegura un error pequeño al resolver cualquier sistema mal condicionado.

9. Pruebas realizadas

Resultado: cinco pruebas automatizadas aprobadas. Comprueban el factor 3×3 conocido y que A se conserve; el caso escalar y la identidad; doce combinaciones generadas (tamaños 2, 7, 16 y 31 con semillas 0, 42 y 2026); entradas inválidas; y la detección de un factor deliberadamente incorrecto.

Entre las entradas rechazadas están las matrices vacías, no cuadradas, asimétricas, indefinidas, semidefinidas sin positividad estricta y aquellas con NaN o infinito. El ejemplo conocido produjo exactamente el factor esperado.

Qué medimos y qué podemos concluir

10. Protocolo de medición

Para cada tamaño n = 32, 64 y 128 usamos la semilla 2026, un calentamiento y siete repeticiones medidas sobre la misma entrada. El reloj monótono **perf_counter** mide la llamada completa a cholesky(): comprobaciones de entrada, reserva de L y factorización. Generación de A, validación y escritura de archivos quedan fuera del cronómetro.

Reportamos mediana, primer y tercer cuartil (Q1 y Q3), y rango intercuartílico IQR = Q3 − Q1. Se usa el método de cuartiles inclusivo de Python. El IQR describe la dispersión observada; no es un intervalo de confianza. La validación entre repeticiones puede modificar la caché.

11. Resultados locales observados

Windows 11 · Python 3.12.14 · 21 de septiembre de 2026. Tiempos en milisegundos; siete observaciones por fila.

n	Mediana	Q1	Q3	IQR	Residuo máx.
32	1.2313	1.2255	1.2627	0.0372	1.08e-16
64	7.3433	7.3293	7.4244	0.0951	9.68e-17
128	55.7097	55.3702	55.9313	0.5611	1.19e-16

Los residuos quedan muy por debajo del criterio establecido. Al duplicar n, las medianas aumentan aproximadamente 5,96 y 7,59 veces. Es compatible con la tendencia cúbica esperada, pero tres tamaños pequeños en Python no demuestran una ley de escalabilidad ni permiten predecir tiempos del clúster.

12. Conclusión y siguiente pregunta

La referencia reconstruye correctamente las matrices ensayadas y el protocolo conserva mediciones individuales y dispersión. Esto respalda los casos probados, no constituye una prueba exhaustiva de estabilidad. Todavía no hay una variante paralela comparable ni un speedup que reportar.

¿Qué pasaría si llevamos el mismo algoritmo a C y separamos los costos de reserva, generación, factorización y validación? Esa pregunta abre la etapa 2. Después estudiaremos cómo agrupar operaciones por bloques y cuáles pueden ejecutarse simultáneamente.

Trazabilidad. Código: desde_cero/cholesky.py; pruebas: test_cholesky.py; medición: benchmark.py. Datos: desde_cero/resultados/20260921T170357862059Z/ (mediciones.csv, resumen.csv y entorno.json). Esta edición utiliza los resultados existentes; no representa una nueva campaña de medición.

Referencia. LAPACK: documentación de DPOTRF. Describe la factorización SPD y una versión de producción por bloques con BLAS de nivel 3. Nuestra referencia es escalar.